Informe de Trabajo - Examen Parcial de Estructuras de Datos

Equipo 1: Farmacia FarmaVida

Integrantes y Métodos Asignados

Integrante	Método Asignado
Jorge Luis Reyes Pérez	MedicamentosAgotados()
Geonel Sánchez	PromedioPrecioPorCategoria(String)
Merlyn Sosa Borges	ActualizarStock(String, int)
Yaisel Pinos Durán	ListarPreciosInversos()
Ray Sanjudo Mesa	AplicarAumentoGeneral(double)

1. Introducción

El presente informe documenta en profundidad el proceso de análisis, diseño, razonamiento e implementación del Ejercicio 1 del Examen Parcial de Estructuras de Datos (Curso 2026-2027). El equipo 1 tiene asignada la problemática del dominio de la farmacia FarmaVida, donde se requiere gestionar de manera eficiente y consistente un inventario dinámico mediante una estructura de Lista Simplemente Enlazada. Cada integrante ha asumido la responsabilidad completa de diseñar y desarrollar un método específico dentro de la lista, asegurando la cohesión y robustez del sistema global.

El objetivo de este documento es dejar constancia detallada y argumentada de:

- La problemática abordada y su contexto operativo en la farmacia.
- La estructura de datos empleada, con una justificación técnica detallada de sus ventajas y limitaciones frente a otras alternativas.
- El razonamiento paso a paso, exhaustivamente argumentado, detrás de cada uno de los métodos asignados.
- El uso transparente y ético declarado de herramientas de Inteligencia Artificial durante la fase de análisis y verificación.
- La reflexión crítica sobre la metodología de trabajo adoptada y las conclusiones académicas del equipo.

2. Problemática: Farmacia FarmaVida

En la farmacia FarmaVida se requiere controlar de forma precisa el inventario de los medicamentos disponibles en stock. El correcto modelado del objeto Medicamento es fundamental para garantizar que las operaciones de cálculo monetario, filtrado y actualización se ejecuten sin errores en tiempo de ejecución.

Atributos de cada medicamento

- **código (String):** Identificador único comercial del producto (ej. "M01"). Es indispensable para realizar búsquedas exactas.
- **nombre (String):** Nombre comercial legible (ej. "Paracetamol").
- **categoría (Categoría enum):** Clasificación cerrada de medicamentos (Analgésico, Antibiótico, Antiinflamatorio, Vitamina). Previene errores tipográficos.
- **stock (int):** Cantidad física disponible en inventario. No se permiten valores negativos.
- **precio (BigDecimal):** Precio unitario expresado en moneda local. Se utiliza BigDecimal en lugar de double para garantizar precisión decimal matemática exacta en cálculos financieros.

Funcionalidades asignadas

- **a) MedicamentosAgotados() — Jorge Luis Reyes Pérez:** Identificación de productos con existencias críticas.
- **b) PromedioPrecioPorCategoria(String categoria) — Geonel Sánchez:** Cálculo de precios promedio por rubro.
- **c) ActualizarStock(String codigo, int cantidadAgregada) — Merlyn Sosa Borges:** Reabastecimiento e incremento de inventario.
- **d) ListarPreciosInversos() — Yaisel Pinos Durán:** Extracción de precios en orden inverso al ingreso.
- **e) AplicarAumentoGeneral(double porcentaje) — Ray Sanjudo Mesa:** Ajuste porcentual masivo de precios.

3. Estructura de Datos Empleada: Lista Simplemente Enlazada

Justificación técnica y argumentación

La elección de una Lista Simplemente Enlazada responde directamente a la naturaleza dinámica del inventario de una farmacia. A diferencia de un arreglo estático de tamaño fijo, la lista enlazada asigna memoria de forma dinámica a medida que se insertan o eliminan nodos, eliminando la necesidad de redimensionar estructuras contiguas en memoria.

Ventajas principales

- **Crecimiento dinámico O(1):** Cada nodo almacena una referencia al dato (Medicamento) y un puntero 'siguiente' al nodo posterior. Esto permite agregar elementos al final o inicio en tiempo constante sin copiar arreglos.
- **Inexistencia de sobrecosto por espacio libre:** Solo se consume memoria para los elementos que realmente existen en el inventario.
- **Adecuación al acceso secuencial:** Dado que todas las funcionalidades requeridas (promedios, aumentos, búsquedas) implican recorrer secuencialmente los productos, la complejidad de recorrido lineal O(n) es computacionalmente óptima y aceptable.

4. Proceso de Razonamiento y Argumentación por Método

Nota metodológica

El equipo adoptó un enfoque riguroso de diseño manual antes de la codificación en Java. Cada miembro dibujó en libreta el comportamiento de la lista, trazando el movimiento de los punteros ('actual', 'anterior', 'siguiente') e identificando proactivamente todos los escenarios borde o de fallo antes de implementar.

4.1. MedicamentosAgotados() - Jorge Luis Reyes Pérez

Argumentación de la estrategia de dos pasadas

El objetivo principal es devolver los nombres de los medicamentos cuyo stock sea menor o igual a 5 unidades. En una lista enlazada, no se conoce de antemano cuántos nodos cumplirán con la condición de filtro.

Dimensionar el arreglo resultado directamente con el tamaño de la lista ('cantidad') es un error grave de diseño. Si solo 2 de 10 medicamentos están agotados, un arreglo de 10 posiciones dejaría 8 posiciones nulas ('null') al final. Esto genera desperdicio de memoria y posibles excepciones de puntero nulo (NullPointerException) al iterar el arreglo resultante.

Por ello, se fundamenta el uso de la estrategia de dos pasadas:

1. **Primera pasada:** Se recorre la lista completa únicamente contando cuántos nodos cumplen 'stock <= 5'.
2. **Creación exacta:** Se instancia el arreglo con la dimensión exacta que devolvió el contador.

3. **Segunda pasada:** Se recorre nuevamente la lista para copiar únicamente los nombres requeridos en las posiciones del arreglo.

Pseudocódigo

```
PROCEDIMIENTO MedicamentosAgotados()
SI (inicio == null) ENTONCES
RETORNAR arreglo vacío
FIN SIcontador = 0

actual = inicio

CICLO (actual != null)

   SI (actual.getStock() <= 5) ENTONCES

      contador++

   FIN SI

   actual = actual.siguiente

FIN CICLO

arreglo = nuevo arreglo de tamaño contador

i = 0

actual = inicio

CICLO (actual != null)

   SI (actual.getStock() <= 5) ENTONCES

      arreglo[i] = actual.getNombre()

      i++

   FIN SI

   actual = actual.siguiente

FIN CICLO
```

RETORNAR arreglo

FIN PROCEDIMIENTO

4.2. PromedioPrecioPorCategoria (String categoria) - Geonel Sánchez

Argumentación técnica

El método debe calcular el precio promedio de todos los medicamentos pertenecientes a una categoría dada, devolviendo 0.0 cuando no existan coincidencias.

Puntos clave de diseño:

- **Prevención de la división por cero:** Si la lista está vacía o si ningún medicamento coincide con la categoría, el contador 'cantidadMedicamentos' finaliza en 0. Ejecutar la división directa provocaría un fallo grave o un resultado indefinido. Por ello, se verifica explícitamente la condición antes de dividir.
- **Comparación exacta enum vs String:** En Java, la categoría del medicamento está tipada como 'Categoría' (enum). Comparar un objeto enum directamente contra un String mediante .equals() devuelve siempre 'false' de forma silenciosa sin error de compilación. La solución correcta exige comparar el nombre técnico de la constante con 'getCategoria().name().equals(categoria)'.
- **Precisión con BigDecimal:** La acumulación se realiza con BigDecimal y la división final especifica 2 decimales y redondeo HALF_UP, asegurando estabilidad monetaria.

Pseudocódigo

PROCEDIMIENTO PromedioPrecioPorCategoria(categoria)
SI (categoria == null) ENTONCES
RETORNAR 0.0
FIN SIactual = inicio

cantidadMedicamentos = 0

sumaPrecios = 0.0

CICLO (actual != null)

SI (actual.getDato().getCategoria().name().equals(categoria)) ENTONCES

sumaPrecios = sumaPrecios + actual.getDato().getPrecio()

```
        cantidadMedicamentos = cantidadMedicamentos + 1

    FIN SI

    actual = actual.siguiente

FIN CICLO

SI (cantidadMedicamentos == 0) ENTONCES

    RETORNAR 0.0

FIN SI

RETORNAR sumaPrecios / cantidadMedicamentos (2 decimales, redondeo HALF_UP)

FIN PROCEDIMIENTO
```

4.3. ActualizarStock(String codigo, int cantidadAgregada) - Merlyn Sosa Borges

Argumentación técnica y manejo de errores

Este método busca un medicamento por su código e incrementa su stock. Devolver 'false' o 'true' resulta ambiguo si no se informa la razón exacta del fallo.

Aspectos críticos de seguridad en el código:

- **Prevención de Desbordamiento de Enteros (Overflow):** Si un medicamento tiene un stock cercano a Integer.MAX_VALUE (2,147,483,647) y se le suma un valor positivo en 'int', se produce un desbordamiento aritmético que convierte el stock en un número negativo corrupto. Para evitarlo, la suma parcial se calcula usando una variable auxiliar de tipo 'long'. Si el valor supera Integer.MAX_VALUE, la operación se rechaza inmediatamente.
- **Claridad informativa:** Ante solicitudes inválidas (código nulo, cantidad de reabastecimiento negativa, desbordamiento de entero o código no encontrado), el método imprime un mensaje explicativo explícito antes de retornar 'false'.

Pseudocódigo

```
PROCEDIMIENTO ActualizarStock(codigo, cantidadAgregada)
SI (codigo == null) ENTONCES
ESCRIBIR "El código es null, se rechaza la solicitud"
```

```
RETORNAR false
FIN SISI (cantidadAgregada < 0) ENTONCES

   ESCRIBIR "La cantidad es negativa, se rechaza la solicitud"

   RETORNAR false

FIN SI

actual = inicio

CICLO (actual != null)

   SI (actual.getCodigo().equals(codigo)) ENTONCES

      aumento = (long) actual.getStock() + cantidadAgregada

      SI (aumento > Integer.MAX_VALUE) ENTONCES

         ESCRIBIR "El stock excede el máximo permitido, se rechaza la solicitud"

         RETORNAR false

      FIN SI

      actual.setStock((int) aumento)

      RETORNAR true

   FIN SI

   actual = actual.siguiente

FIN CICLO

ESCRIBIR "Medicamento no encontrado"

RETORNAR false

FIN PROCEDIMIENTO
```

4.4. ListarPreciosInversos() - Yaisel Pinos Durán

Argumentación del patrón Mutar-Extraer-Restaurar

El requerimiento exige retornar un arreglo con los precios de los medicamentos en el orden inverso en el que fueron insertados.

Fundamentación de la estrategia:

En lugar de crear copias completas de los nodos en memoria auxiliar, se utiliza el patrón de diseño mutar-extraer-restaurar mediante una función auxiliar 'invertir()':

1. **Conteo independiente:** Se realiza una primera iteración para contar los nodos reales existentes ('totalNodos'). No se confía a ciegas en el atributo 'cantidad' de la lista por si este pudiera estar desactualizado.
2. **Primera Inversión:** Se ejecuta 'invertir()', reorientando los punteros 'siguiente' de cada nodo in-place (O(n) en tiempo y O(1) en memoria extra).
3. **Extracción:** Se recorre la lista ahora invertida, guardando secuencialmente cada precio en el arreglo.
4. **Segunda Inversión (Restauración):** Se llama nuevamente a 'invertir()' para reconstruir la lista exactamente a su estado original.

Este enfoque garantiza eficiencia espacial absoluta sin alterar la estructura persistente de los datos.

Pseudocódigo

```
PROCEDIMIENTO invertir()
actual = inicio
anterior = nullCICLO (actual != null)

    siguiente = actual.siguiente

    actual.siguiente = anterior

    anterior = actual

    actual = siguiente

FIN CICLO

inicio = anterior

FIN PROCEDIMIENTO
```

```
PROCEDIMIENTO ListarPreciosInversos()
totalNodos = 0
actual = inicioCICLO (actual != null)

   totalNodos++

   actual = actual.siguiente

FIN CICLO

invertir()

arreglo = nuevo arreglo de tamaño totalNodos

i = 0

actual = inicio

CICLO (actual != null)

   arregl
o[i] = actual.getDato().getPrecio()
i++
actual = actual.siguiente
FIN CICLOinvertir()

RETORNAR arreglo

FIN PROCEDIMIENTO
```

4.5. AplicarAumentoGeneral (double porcentaje) - Ray Sanjudo Mesa

Argumentación técnica sobre tipos de datos y porcentajes

El método incrementa el precio de todos los medicamentos en un porcentaje dado y devuelve la cantidad de registros modificados.

Puntos clave de diseño:

- **Convención del parámetro:** Se establece que el porcentaje se recibe como fracción decimal (ej. 0.10 representa el 10% de aumento, no 10.0).

- **Validación de Números Flotantes no Finitos:** En Java, un valor 'double' puede tomar valores especiales como Double.NaN (Not a Number) o Infinitos. Convertir un NaN o Infinito a BigDecimal provocaría excepciones de formato (NumberFormatException). Por ello, se verifica primero mediante 'Double.isFinite(porcentaje)'.
- **Factor Multiplicador Unificado:** Se calcula 'factor = 1 + porcentaje' una sola vez fuera del ciclo. Se aplica a cada precio mediante 'precio.multiply(factor)', maximizando el rendimiento.
- **Respuestas Informativas:** Porcentajes negativos o cero devuelven 0 e imprimen advertencias aclaratorias sin alterar los datos.

Pseudocódigo

```
PROCEDIMIENTO AplicarAumentoGeneral(porcentaje)
SI (porcentaje no es un número finito) ENTONCES
ESCRIBIR "El porcentaje debe ser un número finito"
RETORNAR 0
FIN SISI (porcentaje < 0) ENTONCES

   ESCRIBIR "El porcentaje no puede ser negativo"

   RETORNAR 0

FIN SI

SI (porcentaje == 0.0) ENTONCES

   ESCRIBIR "No hubo cambios: el porcentaje es 0"

   RETORNAR 0

FIN SI

factor = 1 + porcentaje

actual = inicio

modificados = 0

CICLO (actual != null)

   med = actual.getDato()

   med.setPrecio(med.getPrecio() * factor)
```

```
    modificados++

    actual = actual.siguiente

FIN CICLO

RETORNAR modificados

FIN PROCEDIMIENTO
```

5. Declaración de Uso de Inteligencia Artificial

En estricto cumplimiento con las normativas éticas e indicaciones del curso, el equipo declara de forma transparente el uso de asistentes de IA (tipo Claude, DeepSeek y Gemini) como herramientas de apoyo académico durante el proyecto.

Roles asumidos por la IA

- **Validación de casos borde:** Ayuda en la identificación previa de situaciones complejas (como desbordamiento de enteros en stock o manejo de valores NaN en flotantes).
- **Revisión socrática de pseudocódigo:** Discusión teórica de la lógica diseñada previamente a mano por los integrantes.
- **Asistencia en redacción:** Apoyo en la estructuración y corrección de estilo de la documentación técnica.

La IA no fue utilizada para generar código de forma ciega ni para sustituir el aprendizaje y razonamiento del equipo.

6. Reflexión sobre el Proceso de Trabajo

La metodología de "pensar y trazar a mano antes de programar" demostró ser altamente efectiva. El análisis previo de punteros y de la inmutabilidad de objetos como BigDecimal evitó errores graves que habrían sido difíciles de rastrear en compilación o ejecución. La separación clara de responsabilidades entre los 5 integrantes garantizó un trabajo en equipo armonioso y técnico.

7. Conclusiones

El desarrollo del proyecto permitió afianzar el dominio de las estructuras enlazadas en Java. Se logró un sistema robusto, eficiente e inmune a errores comunes de

desbordamiento, divisiones nulas o inconsistencias de tipos de datos, cumpliendo satisfactoriamente con todos los requerimientos académicos exigidos.